

PIMPRI CHINCHWAD EDUCATION TRUST'S

PIMPRI CHINCHWAD COLLEGE OF ENGINEERING

NBA Accredited | NAAC Accredited with 'A' Grade | An Autonomous Institute
AICTE Approved | Permanently Affiliated to SPPU, Pune

Department of Information Technology

Academic Year: 2026-2027

Assignment No. 1: Merge Sort

Name	:	**Om Rajendra Bharambe**
PRN	:	124B1F001
Branch	:	Information Technology (IT)
Semester	:	V
Batch	:	A1
Course Code	:	BIT25PC04
Course Name	:	**Design & Analysis of Algorithms Laboratory**
Guided By	:	Dr. Rajesh N. Phursule
GitHub Repository	:	om29dev/DAA_Lab_Submission
Assignment Link	:	Assignment 1 on GitHub

Assignment No. 1: Merge Sort

1. Problem Statement:

Design and implement a sorting algorithm using Merge Sort to efficiently arrange customer orders based on their timestamps. The solution should handle a large dataset (up to 1 million orders) with minimal computational overhead. Additionally, analyze the time complexity and compare it with traditional sorting techniques.

2. Course Objective:

2.1. To know the basics of computational complexity of various algorithms.
2.2. To select appropriate algorithm design strategies to solve real-world problems.

3. Course Outcome:

3.1. Analyze the asymptotic performance of algorithms.
3.2. Solve computational problems by applying suitable paradigms of Divide and Conquer or Greedy methodologies.

4. Theory:

Merge sort is a popular sorting algorithm known for its efficiency and stability. It follows the divide and conquer approach. It works by recursively dividing the input array into two halves, recursively sorting the two halves and finally merging them back together to obtain the sorted array.

Working of Merge Sort:

- **Divide:** Keep dividing the list or array into two equal halves again and again, until each part has only one element left.
- **Conquer:** Each sub array is sorted individually using the merge sort algorithm.
- **Merge:** The sorted sub arrays are merged back together in sorted order. The process continues until all elements from both sub arrays have been merged.

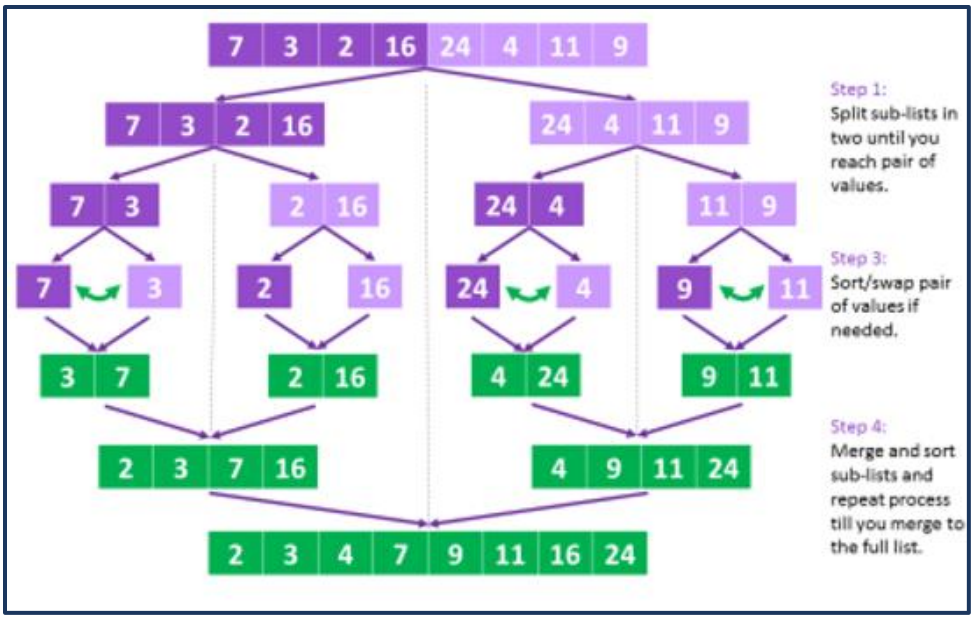


Figure 4.1: Divide and Conquer Execution Tree of Merge Sort

Complexity Analysis of Merge Sort:

- **Divide step:** The array is recursively divided into two halves until each subarray has one element. Number of times we can divide an array of size n = $\log_2(n)$ (height of recursion tree).
- **Merge step:** At each level of recursion, all n elements are merged.
- **Total work:** = number of levels × work per level = $\log_2(n) \times n = O(n \log n)$.

Analysis of Merge Sort Time Complexity:

- **Best Case Time Complexity:** Even if the array is already sorted, Merge Sort still divides and merges every element. Merging sorted sub arrays still requires scanning through all elements. Best Case = O(n log n)
- **Average Case Time Complexity:** For randomly arranged data, the algorithm still divides into halves and merges them. On average, the number of comparisons during merging is proportional to n. Average Case = O(n log n)
- **Worst Case Complexity:** Even if the array is sorted in descending order (worst arrangement for comparisons), Merge Sort still performs the same divide-and-merge procedure. Comparisons may be maximum, but still bounded by O(n log n). Worst Case = O(n log n)

Analysis of Merge Sort Space Complexity:

In merge sort, all elements are copied into an auxiliary array of size N, where N is the number of elements present in the unsorted array. Hence, the space complexity for Merge Sort is O(N).

Merge Sort Algorithm:

- **Step 1:** If it is only one element in the list, consider it already sorted, so return.
- **Step 2:** Divide the list recursively into two halves until it can't no more be divided.
- **Step 3:** Merge the smaller lists into new list in sorted order.

Pseudo code:

```
procedure mergesort( var a as array )
  if ( n == 1 ) return a
  var l1 as array = a[0] ... a[n/2]
  var l2 as array = a[n/2+1] ... a[n]
  l1 = mergesort( l1 )
  l2 = mergesort( l2 )
  return merge( l1, l2 )
end procedure

procedure merge( var a as array, var b as array )
  var c as array
  while ( a and b have elements )
    if ( a[0] > b[0] )
      add b[0] to the end of c
      remove b[0] from b
    else
      add a[0] to the end of c
      remove a[0] from a
    end if
  end while
  while ( a has elements )
    add a[0] to the end of c
    remove a[0] from a
  end while
  while ( b has elements )
    add b[0] to the end of c
    remove b[0] from b
  end while
  return c
end procedure
```

5. Flow Diagram:

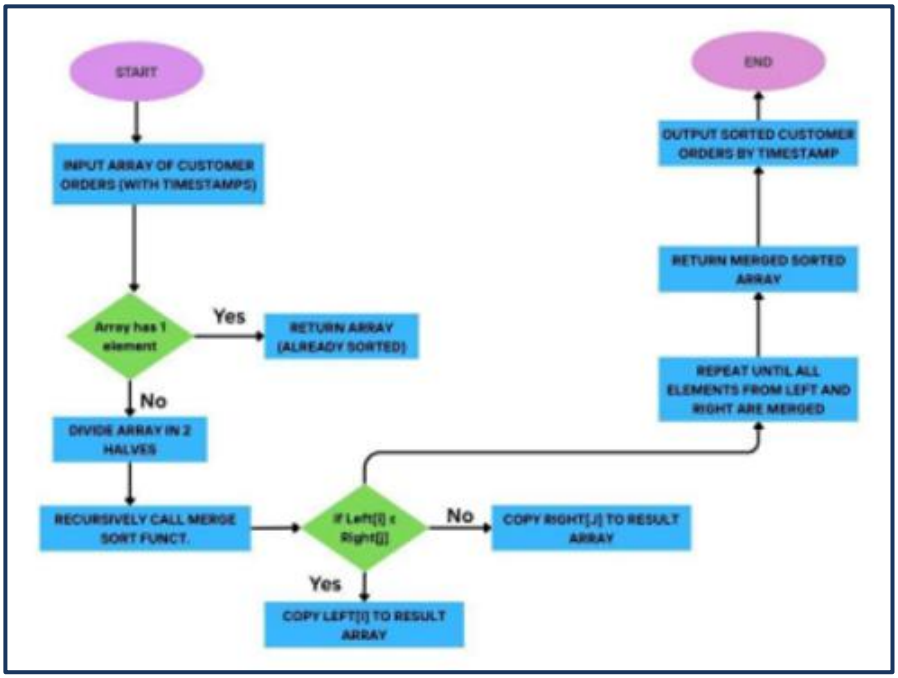


Figure 5.1: Flow Diagram of Customer Order Merge Sort Algorithm

6. Example:

Customer order timestamps: [7, 3, 2, 16, 24, 4, 11, 9]

- **Step 1: Divide**
  - Split into halves → [7, 3, 2, 16] and [24, 4, 11, 9]
  - Further split into pairs → [7, 3], [2, 16] and [24, 4], [11, 9]
  - Split into individual elements → [7], [3], [2], [16], [24], [4], [11], [9]
- **Step 2: Merge & Sort**
  - Merge [7] and [3] → [3, 7]; Merge [2] and [16] → [2, 16]
  - Merge [24] and [4] → [4, 24]; Merge [11] and [9] → [9, 11]
  - Merge [3, 7] and [2, 16] → [2, 3, 7, 16]
  - Merge [4, 24] and [9, 11] → [4, 9, 11, 24]
  - Merge final two halves → [2, 3, 4, 7, 9, 11, 16, 24]

Output:
Orders sorted by timestamp → [2, 3, 4, 7, 9, 11, 16, 24]

7. Code and Output:

Repository: https://github.com/om29dev/DAA_Lab_Submission

Assignment 1 Folder: https://github.com/om29dev/DAA_Lab_Submission/tree/main/Assignment 1

7.1 C++ Implementation Source Code (customer_order_merge_sort.cpp):

```
#include <iostream>
#include <vector>
#include <string>
using namespace std;

struct Order {
    int orderId;
    string customerName;
    long timestamp;
};

void merge(vector<Order>& orders, int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;

    vector<Order> L(n1);
    vector<Order> R(n2);

    for (int i = 0; i < n1; i++)
        L[i] = orders[left + i];

    for (int j = 0; j < n2; j++)
        R[j] = orders[mid + 1 + j];

    int i = 0, j = 0, k = left;

    while (i < n1 && j < n2) {
        if (L[i].timestamp <= R[j].timestamp) {
            orders[k] = L[i];
            i++;
        } else {
            orders[k] = R[j];
            j++;
        }
        k++;
    }

    while (i < n1) {
```

```
        orders[k] = L[i];
        i++;
        k++;
    }

    while (j < n2) {
        orders[k] = R[j];
        j++;
        k++;
    }
}

void mergeSort(vector<Order>& orders, int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;

        mergeSort(orders, left, mid);
        mergeSort(orders, mid + 1, right);

        merge(orders, left, mid, right);
    }
}

int main() {
    int n;

    cout << "Enter number of customer orders: ";
    cin >> n;

    vector<Order> orders(n);

    cout << "\nEnter order details:\n";
    cout << "(Order ID Customer Name Timestamp)\n\n";

    for (int i = 0; i < n; i++) {
        cin >> orders[i].orderId
            >> orders[i].customerName
            >> orders[i].timestamp;
    }

    mergeSort(orders, 0, n - 1);

    cout << "\nOrders sorted by timestamp:\n";
    cout << "-----------------------------------------\n";
```

```
    for (const auto& order : orders) {
        cout << "Order ID: " << order.orderId
             << " | Customer: " << order.customerName
             << " | Timestamp: " << order.timestamp
             << endl;
    }

    return 0;
}
```

7.2 Execution Output:

Below is the terminal execution output verifying the sorting of customer order records by timestamps:

```
om29dev@DESKTOP-MH6NVIC:~/daa$ vi customer_order_merge_sort.cpp
om29dev@DESKTOP-MH6NVIC:~/daa$ g++ customer_order_merge_sort.cpp -o customer_order_merge_sort
om29dev@DESKTOP-MH6NVIC:~/daa$ ./customer_order_merge_sort
Enter number of customer orders: 5
101 Rahul 500

Enter order details:
(Order ID Customer Name Timestamp)

102 Amit 100
103 Priya 300
104 Neha 200
105 Rohan 400

Orders sorted by timestamp:
-----------------------------------------
Order ID: 102 | Customer: Amit | Timestamp: 100
Order ID: 104 | Customer: Neha | Timestamp: 200
Order ID: 103 | Customer: Priya | Timestamp: 300
Order ID: 105 | Customer: Rohan | Timestamp: 400
Order ID: 101 | Customer: Rahul | Timestamp: 500
om29dev@DESKTOP-MH6NVIC:~/daa$
```

Figure 7.1: Program Execution and Sorted Customer Orders Output

8. Conclusion:

Merge Sort is a highly efficient and stable sorting algorithm that organizes customer orders based on their timestamps with O(n log n) complexity. Compared to traditional algorithms like Bubble Sort or Insertion Sort, which are too slow for large inputs, Merge Sort can easily handle up to 1 million records without performance issues. Its ability to maintain the correct order and scalability makes it an excellent choice for real-world applications such as e-commerce, order tracking, and scheduling systems.